Sujet 1: Code Smell Detection via Pearson Correlation and ML Hyper

Abstract : This study addresses the challenge of detecting code smells in large-scale software systems using machine learning (ML). Traditional detection methods often suffer from low accuracy and poor generalization across different datasets. To overcome these issues, we propose a machine learning-based model that automatically and accurately identifies code smells, offering a scalable solution for software quality analysis. The novelty of our approach lies in the use of eight diverse ML algorithms, including XGBoost, AdaBoost, and other classifiers, alongside key techniques such as the Synthetic Minority Over-sampling Technique (SMOTE) for class imbalance and Pearson correlation for efficient feature selection. These methods collectively improve model accuracy and generalization. Our methodology involves several steps: first, we preprocess the data and apply SMOTE to balance the dataset; next, Pearson correlation is used for feature selection to reduce redundancy; followed by training eight ML algorithms and tuning hyperparameters through Grid Search, Random Search, and Bayesian Optimization. Finally, we evaluate the models using accuracy, F-measure, and confusion matrices. The results show that AdaBoost, Random Forest, and XGBoost perform best, achieving accuracies of 100%, 99%, and 99%, respectively. This study provides a robust framework for detecting code smells, enhancing software quality assurance, and demonstrating the effectiveness of a comprehensive, optimized ML approach.

Keywords: Code smell detection, Software quality assurance, Machine learning algorithms, Hyperparameter optimization, Random Forest, AdaBoost, K-Nearest Neighbor (KNN), Software maintainability, Classifi-
cation performance, Software engineering

Sujet 2: An Empirical Study on the Classification of Bug Reports with Machine Learning

Abstract: Software defects are a major threat to the reliability of computer systems. The literature shows that more than 30% of bug reports submitted in large software projects are misclassified (i.e., are feature requests, or mistakes made by the bug reporter), leading developers to place great effort in manually inspecting them. Machine Learning algorithms can be used for the automatic classification of issue reports. Still, little is known regarding key aspects of training models, such as the influence of programming languages and issue tracking systems. In this paper, we use a dataset containing more than 660,000 issue reports, collected from heterogeneous projects hosted in different issue tracking systems, to study how different factors (e.g., project language, report content) can influence the performance of models in handling classification of issue reports. Results show that using the report title or description does not significantly differ; Support Vector Machine, Logistic Regression, and

Random Forest are effective in classifying issue reports; programming languages and issue tracking systems influence classification outcomes; and models based on heterogeneous projects can classify reports from projects not present during training. Based on findings, we propose guidelines for future research, including recommendations for using heterogeneous data and selecting high-performing algorithms.

Lien arXiv :https://arxiv.org/abs/2503.00660

Code et données : https://zenodo.org/records/7377402

Sujet 3 : Improving Code Generation via Small Language Model-as-a-judge

abstract : Large language models (LLMs) have shown remarkable capabilities in automated code generation. While effective for mainstream languages, they may underperform on less common or domain-specific languages, prompting companies to develop in-house code generators. While open-source models can be trained for this, only LLMs with tens of billions of parameters match the performance of commercial tools, demanding costly training and deployment. Recent work proposed supporting code generation with smaller models (SLMs) by generating multiple candidate solutions and using another SLM to select the most likely correct one. The most recent work in this area is the one by Sun et al. [29] presenting RankEF, a T5 model trained to rank code solutions using both execution-based and non-execution-based information. However, Sun et al. do not assess the T5 ranker's classification accuracy, that is, how often it misjudges correct implementations as incorrect or vice versa, leaving open questions about the reliability of LMs as code correctness judges for other tasks (e.g., automated code review). Moreover, their experiments involve relatively old models, making it unclear the extent to which such a methodology would still help companies in cheaply training their own code generators with performance comparable to those of massive LLMs. We present a study addressing these limitations. We train several state-of-the-art SLMs as code correctness judges and assess their ability to discriminate between correct and wrong implementations. We show that modern SLMs outperform RankEF, even without exploiting execution-based information. When used as code rankers, they achieve higher performance gains than RankEF and perform competitively with LLMs 5-25x larger, at a fraction of the cost.

Sujet 4: RawMal-TF: Raw Malware Dataset Labeled by Type and Family

Abstract: This work addresses the challenge of malware classification using machine learning by developing a novel dataset labeled at both the malware type and family levels. Raw binaries were collected from sources such as VirusShare, VX Underground, and MalwareBazaar, and subsequently labeled with family information parsed from binary names

and type-level labels integrated from ClarAVy. The dataset includes 14 malware types and 17 malware families, and was processed using a unified feature extraction pipeline based on static analysis, particularly extracting features from Portable Executable headers, to support advanced classification tasks. The evaluation was focused on three key classification tasks. In the binary classification of malware versus benign samples, Random Forest and XGBoost achieved high accuracy on the full datasets, reaching 98.5% for type-based detection and 98.98% for family-based detection. When using truncated datasets of 1,000 samples to assess performance under limited data conditions, both models still performed strongly, achieving 97.6% for type-based detection and 98.66% for family-based detection. For interclass classification, which distinguishes between malware types or families, the models reached up to 97.5% accuracy on type-level tasks and up to 93.7% on family-level tasks. In the multiclass classification setting, which assigns samples to the correct type or family, SVM achieved 81.1% accuracy on type labels, while Random Forest and XGBoost reached approximately 73.4% on family labels. The results highlight practical trade-offs between accuracy and computational cost, and demonstrate that labeling at both the type and family levels enables more fine-grained and insightful malware classification. The work establishes a robust foundation for future research on advanced malware detection and classification.

Lien arXiv : https://arxiv.org/abs/2506.23909
Code et données : https://github.com/CS-and-AI/RawMal-TF